BookSwap mit SQLite

13 Tabellen, 53 Attribute, 14 Fremdschlüssel

Katalog, Exemplare und Ausleihen

196 synthetische Einträge.
Mindestens zehn Datensätze je Tabelle.
24 Exemplare, zwölf Ausleihen, zwei Views.

DLBDSPBDM01_D

Überarbeitete Endfassung 15.10.2026

Logo: IU Internationale Hochschule, n.d. Eigener Daten- und Prüfstand 03.10.2026.

Abbildungsverzeichnis

Abb. 1: adresse (Folie 4)
Abb. 2: benutzer (Folie 5)
Abb. 3: verlag (Folie 6)
Abb. 4: sprache (Folie 7)
Abb. 5: buch (Folie 8)
Abb. 6: autor (Folie 9)
Abb. 7: buch_autor (Folie 10)
Abb. 8: genre (Folie 11)
Abb. 9: buch_genre (Folie 12)
Abb. 10: exemplar (Folie 13)
Abb. 11: abholzeit (Folie 14)
Abb. 12: ausleihe (Folie 15)
Abb. 13: bewertung (Folie 16)

Zusammenfassung der Implementierung

BookSwap überführt das Konzept in eine reproduzierbare SQLite-Datenbank. 13 Tabellen mit 53 Attributen und 14 Fremdschlüsseln trennen Buchausgaben von physischen Exemplaren. Autoren und Genres verwenden Zuordnungstabellen. Die 196 synthetischen Einträge beschreiben keine realen Ausleihvorgänge. Jede Tabelle enthält mindestens zehn Datensätze. Die gemeinsame SQL-Datei erstellt Schema, Trigger, Indizes, Testdaten und zwei Views.

Pflichtfelder, Eindeutigkeit, Wertebereiche und Fremdschlüssel sichern die Daten. STRICT prüft die Datentypen. Trigger verhindern Selbstausleihen, Überschneidungen, zu lange vereinbarte Leihfristen, Bewertungen vor Rückgabe sowie unzulässige Abholung oder Versand. Eine verspätete tatsächliche Rückgabe bleibt erfassbar. Abholfenster verwenden ein eindeutiges Tripel aus Exemplar, Beginn und Ende. Zeitwerte folgen der UTC-Konvention. Fremdschlüssel und rekursive Trigger werden pro Verbindung vor der Transaktion aktiviert.

Drei Join-Pfade verbinden Bücher mit Autoren, Exemplare mit Adressen und Benutzer mit Ausleihen. Sechs Beispielabfragen und beide Views wurden im ursprünglichen Stand ausgeführt. Die Verfügbarkeitsansicht liefert 21 Exemplare. Die Nähesuche berücksichtigt nur angebotene Exemplare ohne laufende Ausleihe. Sie nutzt Haversine mit 6.371 km Erdradius und benötigt SQLite-Mathematikfunktionen. Am Testmittelpunkt 52,52° N und 13,405° E ergeben 2 km Radius sechs Treffer. Zusatzabschnitte für Abfragen, Tests und Benchmarks stehen als Kommentarblöcke in der SQL-Abgabedatei und werden getrennt ausgeführt. Benchmark-Daten gehören in eine Testkopie.

SQLite, 2025b, Abschn. 2/4. SQLite, 2026c, Abschn. 2. SQLite, n.d., Abschn. 6. SQLite, 2026a, Abschn. 1. SQLite, 2026b, Abschn. 1–3.

Ursprüngliche Ergebnisse und Grenzen

155 Tests bestanden, 0 fehlgeschlagen

56 Suchvergleiche prüfen auch Pole, Datumsgrenze und Nullradius.

14 Tage erlaubt, eine Sekunde mehr und Selbstausleihe abgewiesen.
Bewertung erst nach Rückgabe. Versand erfordert eine Freigabe.

Tab. 1: SQLite-VM-Schritte auf einer separaten Testkopie

Abfrage	Ohne Index	Mit Index
Personenhistorie	45.876	5.471
Kartenfenster	50.180	150
Nähesuche	327.182	8.169

Identische, nichtleere Ergebnisse. Die Werte erlauben keine allgemeine Leistungszusage.

Damals fehlten App, Anmeldung, Reservierungen und Angebotsversionen.

Quelle: Eigene Prüfung und Messung vom 03.10.2026, SQLite 3.45.3. SQLite, 2025a, Abschn. 1.1–1.2. SQLite, 2022, Abschn. 2.1.

adresse

Orte und gültige Kartenkoordinaten

SQL-Erstellung (Auszug)

```
CREATE TABLE adresse (
adresse_id INTEGER PRIMARY KEY,
latitude REAL NOT NULL CHECK (latitude BETWEEN -90 AND 90),
-- Weitere Spalten: vollständige SQL-Datei.
```

Vollständige Testabfrage

```
SELECT adresse_id, ort, latitude, longitude,
COUNT(*) OVER() AS anzahl
FROM adresse ORDER BY adresse_id LIMIT 5;
```

Abb. 1. adresse: 12 Datensätze, fünf Stichprobenzeilen

adresse_id	ort	latitude	longitude	anzahl
1	Teststadt	52.52	13.405	12
2	Teststadt	52.525	13.41	12
3	Teststadt	52.53	13.415	12
4	Teststadt	52.54	13.43	12
5	Teststadt	52.6	13.5	12

Quelle: Eigene DBMS-Aufnahme (Spaltenausschnitte). SQL: separate Abgabedatei, 01_schema und 08_screenshot_queries.

benutzer

Eindeutige E-Mail und Zuordnung zur Wohnadresse

SQL-Erstellung (Auszug)

```
CREATE TABLE benutzer (
benutzer_id INTEGER PRIMARY KEY,
adresse_id INTEGER NOT NULL REFERENCES adresse(adresse_id) ON
 DELETE RESTRICT ON UPDATE RESTRICT
```

Vollständige Testabfrage

```
SELECT benutzer_id, name, email, adresse_id,
COUNT(*) OVER() AS anzahl
FROM benutzer ORDER BY benutzer_id LIMIT 5;
```

Abb. 2. benutzer: 12 Datensätze, fünf Stichprobenzeilen

benutzer_id	name	email
1	Mira Test	mira@example.invalid
2	Noah Test	noah@example.invalid
3	Alina Test	alina@example.invalid
4	Emil Test	emil@example.invalid
5	Lina Test	lina@example.invalid

adresse_id	anzahl
1	12
2	12
3	12
4	12
5	12

Quelle: Eigene DBMS-Aufnahme (Spaltenausschnitte). SQL: separate Abgabedatei, 01_schema und 08_screenshot_queries.

verlag

Verlage getrennt von bibliografischen Buchausgaben

SQL-Erstellung

```
CREATE TABLE verlag (
verlag_id INTEGER PRIMARY KEY,
name TEXT NOT NULL CHECK (name = trim(name) AND length(name)>0)
) STRICT;
```

Vollständige Testabfrage

```
SELECT verlag_id, name,
COUNT(*) OVER() AS anzahl
FROM verlag ORDER BY verlag_id LIMIT 5;
```

Abb. 3. verlag: 10 Datensätze, fünf Stichprobenzeilen

verlag_id	name
1	Demo Nordlicht
2	Demo Seitenwind
3	Demo Papierwald
4	Demo Wortstrom
5	Demo Leseraum

anzahl
10
10
10
10
10

Quelle: Eigene DBMS-Aufnahme (Spaltenausschnitte). SQL: separate Abgabedatei, 01_schema und 08_screenshot_queries.

sprache

Eindeutige Sprachcodes und Katalogbezeichnungen

SQL-Erstellung (Auszug)

```
CREATE TABLE sprache (
sprache_id INTEGER PRIMARY KEY,
code TEXT NOT NULL UNIQUE CHECK (code = trim(code) AND
  length(code)>0),
```

Vollständige Testabfrage

```
SELECT sprache_id, code, name,
COUNT(*) OVER() AS anzahl
FROM sprache ORDER BY sprache_id LIMIT 5;
```

Abb. 4. sprache: 10 Datensätze, fünf Stichprobenzeilen

sprache_id	code
1	de
2	en
3	fr
4	es
5	it

name	anzahl
Deutsch	10
Englisch	10
Französisch	10
Spanisch	10
Italienisch	10

Quelle: Eigene DBMS-Aufnahme (Spaltenausschnitte). SQL: separate Abgabedatei, 01_schema und 08_screenshot_queries.

buch

Bibliografische Ausgabe mit Verlag und Sprache

SQL-Erstellung (Auszug)

```
CREATE TABLE buch (
buch_id INTEGER PRIMARY KEY,
jahr INTEGER NOT NULL CHECK (jahr BETWEEN 1 AND 9999),
sprache_id INTEGER NOT NULL REFERENCES sprache(sprache_id) ON
  DELETE RESTRICT ON UPDATE RESTRICT
```

Vollständige Testabfrage

```
SELECT buch_id, titel, jahr, verlag_id, sprache_id,
COUNT(*) OVER() AS anzahl
FROM buch ORDER BY buch_id LIMIT 5;
```

Abb. 5. buch: 12 Datensätze, fünf Stichprobenzeilen

buch_id	titel	jahr
1	Der Garten am Testfluss	2020
2	A Map of Paper	2021
3	La maison des pages	2022
4	El camino del libro	2023
5	Il giardino di carta	2020

verlag_id	sprache_id	anzahl
1	1	12
2	2	12
3	3	12
4	4	12
5	5	12

Quelle: Eigene DBMS-Aufnahme (Spaltenausschnitte). SQL: separate Abgabedatei, 01_schema und 08_screenshot_queries.

autor

Stabile Autoren-ID, gleiche Namen bleiben möglich

SQL-Erstellung

```
CREATE TABLE autor (
autor_id INTEGER PRIMARY KEY,
name TEXT NOT NULL CHECK (name = trim(name) AND length(name)>0)
) STRICT;
```

Vollständige Testabfrage

```
SELECT autor_id, name,
COUNT(*) OVER() AS anzahl
FROM autor ORDER BY autor_id LIMIT 5;
```

Abb. 6. autor: 10 Datensätze, fünf Stichprobenzeilen

autor_id	name
1	Ari Muster
2	Bo Muster
3	Cleo Muster
4	Dara Muster
5	Eli Muster

anzahl
10
10
10
10
10

Quelle: Eigene DBMS-Aufnahme (Spaltenausschnitte). SQL: separate Abgabedatei, 01_schema und 08_screenshot_queries.

buch_autor

n:m-Zuordnung zwischen Buchausgaben und Autoren

SQL-Erstellung (Auszug)

```
CREATE TABLE buch_autor (
buch_id INTEGER NOT NULL REFERENCES buch(buch_id) ON DELETE
  RESTRICT ON UPDATE RESTRICT,
autor_id INTEGER NOT NULL REFERENCES autor(autor_id) ON DELETE
  RESTRICT ON UPDATE RESTRICT,
PRIMARY KEY (buch_id, autor_id)
```

Vollständige Testabfrage

```
SELECT buch_id, autor_id,
COUNT(*) OVER() AS anzahl
FROM buch_autor ORDER BY buch_id, autor_id LIMIT 5;
```

Abb. 7. buch_autor: 13 Datensätze, fünf Stichprobenzeilen

buch_id	autor_id
1	1
1	2
2	2
3	3
4	4

anzahl
13
13
13
13
13

Quelle: Eigene DBMS-Aufnahme (Spaltenausschnitte). SQL: separate Abgabedatei, 01_schema und 08_screenshot_queries.

genre

Wiederverwendbare, eindeutig benannte Kategorien

SQL-Erstellung

```
CREATE TABLE genre (
genre_id INTEGER PRIMARY KEY,
name TEXT NOT NULL UNIQUE CHECK (name = trim(name) AND
  length(name)>0)
) STRICT;
```

Vollständige Testabfrage

```
SELECT genre_id, name,
COUNT(*) OVER() AS anzahl
FROM genre ORDER BY genre_id LIMIT 5;
```

Abb. 8. genre: 10 Datensätze, fünf Stichprobenzeilen

genre_id
1
2
3
4
5

name	anzahl
Roman	10
Krimi	10
Fantasy	10
Sachbuch	10
Lyrik	10

Quelle: Eigene DBMS-Aufnahme (Spaltenausschnitte). SQL: separate Abgabedatei, 01_schema und 08_screenshot_queries.

buch_genre

Mehrere Genres je Ausgabe, jedes Paar nur einmal

SQL-Erstellung (Auszug)

```
CREATE TABLE buch_genre (
buch_id INTEGER NOT NULL REFERENCES buch(buch_id) ON DELETE
  RESTRICT ON UPDATE RESTRICT,
genre_id INTEGER NOT NULL REFERENCES genre(genre_id) ON DELETE
  RESTRICT ON UPDATE RESTRICT,
PRIMARY KEY (buch_id, genre_id)
```

Vollständige Testabfrage

```
SELECT buch_id, genre_id,
COUNT(*) OVER() AS anzahl
FROM buch_genre ORDER BY buch_id, genre_id LIMIT 5;
```

Abb. 9. buch_genre: 13 Datensätze, fünf Stichprobenzeilen

buch_id	genre_id
1	1
1	6
2	2
3	3
4	4

anzahl
13
13
13
13
13

Quelle: Eigene DBMS-Aufnahme (Spaltenausschnitte). SQL: separate Abgabedatei, 01_schema und 08_screenshot_queries.

exemplar

Physisches Exemplar mit Eigentümer und damaligen Angebotsbedingungen

SQL-Erstellung (Auszug)

```
CREATE TABLE exemplar (
exemplar_id INTEGER PRIMARY KEY,
max_tage INTEGER NOT NULL CHECK (max_tage BETWEEN 1 AND 365),
versand INTEGER NOT NULL DEFAULT 0 CHECK (versand IN (0, 1)),
```

Vollständige Testabfrage

```
SELECT exemplar_id, buch_id, eigentuemer_id, versand,
  angeboten,
COUNT(*) OVER() AS anzahl
FROM exemplar ORDER BY exemplar_id LIMIT 5;
```

Abb. 10. exemplar: 24 Datensätze, fünf Stichprobenzeilen

exemplar_id	buch_id	eigentuemer_id
1	1	1
2	2	2
3	3	3
4	4	4
5	5	5

versand	angeboten	anzahl
0	1	24
1	1	24
0	1	24
1	1	24
0	1	24

Quelle: Eigene DBMS-Aufnahme (Spaltenausschnitte). SQL: separate Abgabedatei, 01_schema und 08_screenshot_queries.

abholzeit

Eindeutiges Tripel und halboffenes Abholfenster [von, bis)

SQL-Erstellung (Auszug)

```
CREATE TABLE abholzeit (
abholzeit_id INTEGER PRIMARY KEY,
exemplar_id INTEGER NOT NULL REFERENCES exemplar(exemplar_id) ON
  DELETE RESTRICT ON UPDATE RESTRICT,
UNIQUE (exemplar_id, von, bis)
```

Vollständige Testabfrage

```
SELECT abholzeit_id, exemplar_id, von, bis,
COUNT(*) OVER() AS anzahl
FROM abholzeit ORDER BY abholzeit_id LIMIT 5;
```

Abb. 11. abholzeit: 48 Datensätze, fünf Stichprobenzeilen

abholzeit_id	exemplar_id	von	bis	anzahl
1	1	2026-09-01 09:00:00	2026-09-01 18:00:00	48
2	2	2026-09-01 09:00:00	2026-09-01 18:00:00	48
3	3	2026-09-01 09:00:00	2026-09-01 18:00:00	48
4	4	2026-09-01 09:00:00	2026-09-01 18:00:00	48
5	5	2026-09-01 09:00:00	2026-09-01 18:00:00	48

Quelle: Eigene DBMS-Aufnahme (Spaltenausschnitte). SQL: separate Abgabedatei, 01_schema und 08_screenshot_queries.

ausleihe

Vereinbarte Fälligkeit und tatsächliche Rückgabe getrennt

SQL-Erstellung (Auszug)

```
CREATE TABLE ausleihe (
ausleihe_id INTEGER PRIMARY KEY,
entleiher_id INTEGER NOT NULL REFERENCES benutzer(benutzer_id)
  ON DELETE RESTRICT ON UPDATE RESTRICT,
```

Vollständige Testabfrage

```
SELECT ausleihe_id, exemplar_id, entleiher_id, weg, rueckgabe,
COUNT(*) OVER() AS anzahl
FROM ausleihe ORDER BY ausleihe_id LIMIT 5;
```

Abb. 12. ausleihe: 12 Datensätze, fünf Stichprobenzeilen

ausleihe_id	exemplar_id	entleiher_id
1	1	2
2	2	3
3	3	4
4	4	5
5	5	6

weg	rueckgabe	anzahl
abholung	2026-09-16 10:00:00	12
post	2026-09-10 10:00:00	12
abholung	2026-09-10 10:00:00	12
post	2026-09-10 10:00:00	12
abholung	2026-09-10 10:00:00	12

Quelle: Eigene DBMS-Aufnahme (Spaltenausschnitte). SQL: separate Abgabedatei, 01_schema und 08_screenshot_queries.

bewertung

Eine Bewertung je abgeschlossener Ausleihe, ein bis fünf Sterne

SQL-Erstellung (Auszug)

```
CREATE TABLE bewertung (
bewertung_id INTEGER PRIMARY KEY,
sterne INTEGER NOT NULL CHECK (sterne BETWEEN 1 AND 5),
kommentar TEXT
```

Vollständige Testabfrage

```
SELECT bewertung_id, ausleihe_id, sterne,
COUNT(*) OVER() AS anzahl
FROM bewertung ORDER BY bewertung_id LIMIT 5;
```

Abb. 13. bewertung: 10 Datensätze, fünf Stichprobenzeilen

bewertung_id	ausleihe_id
1	1
2	2
3	3
4	4
5	5

sterne	anzahl
5	10
4	10
3	10
5	10
4	10

Quelle: Eigene DBMS-Aufnahme (Spaltenausschnitte). SQL: separate Abgabedatei, 01_schema und 08_screenshot_queries.

Literatur

IU Internationale Hochschule. (n.d.).
IU Internationale Hochschule [Logo].
https://www.iu.de/iu-logo.svg

SQLite. (n.d.).
CREATE TRIGGER.
https://www.sqlite.org/lang_createtrigger.html

SQLite. (2022, 18. April).
Partial indexes.
https://www.sqlite.org/partialindex.html

SQLite. (2025a, 31. Mai).
EXPLAIN QUERY PLAN.
https://www.sqlite.org/eqp.html

SQLite. (2025b, 12. Juni).
STRICT tables.
https://www.sqlite.org/stricttables.html

SQLite. (2026a, 2. Februar).
Built-in mathematical SQL functions.
https://www.sqlite.org/lang_mathfunc.html

SQLite. (2026b, 11. Juli).
Date and time functions.
https://www.sqlite.org/lang_datefunc.html

SQLite. (2026c, 20. März).
SQLite foreign key support.
https://www.sqlite.org/foreignkeys.html